Practicum Statements Regarding Integers., Simple Induction Principle., Extended Induction Principle, Strong Induction Principle., General Form of Induction, Algorithms., Notation for Algorithms., Some Examples of Algorithms., Integers.

WEEK 15

Subject:

Computational Mathematics

Prepared by:

Aisyah Currents

25091397108

Class International/Number 2

INFORMATICS MANAGEMENT STUDY PROGRAM

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2025

Table of Contents

PRACTICUM Part 1: Applications of Statements Regarding Integers., Simple Induction Principle., Extended Induction Principle.

1. Statements Regarding Integers

Mathematical induction is a proof method that is often used to show that a statement holds for all integers. This practicum aims to understand the concept of mathematical induction and apply it with Python through simple simulations and implementations.

Statements about whole numbers are often expressed in formula form.

• The sum of the first n whole numbers

$Sn=1+2+3+\cdots+n = (n(n+1))/2$

• The sum of the squares of the first n whole numbers

$Qn=12+22+32+\cdots+n2 = (n(n+1)(2n+1))/6$

Prove that the sum of the first n whole numbers is $Sn=(n(n+1))/2$

The Source Code with output

part1.py

```python
# Number 1
def sum_of_integers(n):
    # Mathematical formula
    return n * (n + 1) // 2

# Validation for n = 10
n = 10
calculated = sum_of_integers(n)
actual = sum(range(1, n + 1))

print(f"Sum of the first {n} integers: ")
print(f"Formula result: {calculated}, Direct calculation: {actual}")
print("Valid" if calculated == actual else "Not Valid")

```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Pythor
rams/Python/Python313/python.exe "c:/Users/USER/One
1.py"
Sum of the first 10 integers:
Formula result: 55, Direct calculation: 55
Valid
PS C:\Users\USER\OneDrive\Documents\Currents\Pythor
```

The Analysis

This code computes the sum of the first n integers in two ways,using the arithmetic series formula n(n+1)/2 and by direct iteration then compares the results to validate the calculation; when run for n=10, both methods return 55, and the script prints "Valid," demonstrating a quick correctness check between a closed-form solution and a loop-based accumulation.

2. Simple Induction Principle

This principle involves three steps:

1. **Base Case**

   Show that the statement is true for n=1.

2. **Inductive Hypothesis**

   Assume the statement is true for n=k.

3. **Inductive Step**

   Prove the statement for n=k+1 using the inductive hypothesis.

Prove the following formula:

Sn=n(n+1)/2

The Source Code with output

```
15  # Number 2
16  def validate_induction_simple(n_max):
17
18      for n in range(1, n_max + 1):
19          calculated = n * (n + 1) // 2
20          actual = sum(range(1, n + 1))
21
22          if calculated != actual:
23              return f"Induction failed at n = {n}"
24
25      return "Induction succeeded for all n up to " + str(n_max)
26
27  # Validation up to n = 10
28  print(validate_induction_simple(10))
29
```

```
PS C:\Users\USER\OneDrive\Documents\Current
& C:/Users/USER/AppData/Local/Programs/Pyth
/Matematika Komputasi/Week 15/part1.py"
Induction succeeded for all n up to 10
```

The Source Code with output

```python
# Number 3
def validate_induction_simple(n_max):

    for n in range(1, n_max + 1):
        calculated = n * (n + 1) // 2
        actual = sum(range(1, n + 1))

        if calculated != actual:
            return f"Induction failed at n = {n}"

    return "Induction succeeded for all n up to " + str(n_max)

# Validation up to n = 10
print(validate_induction_simple(10))
```

```
& C:/Users/USER/AppData/Local/Programs/Pyt
/Matematika Komputasi/Week 15/part1.py"
Induction succeeded for all n up to 10
```

The Analysis

The code demonstrates the validation of the arithmetic series formula Sn=n(n+1)/2 using mathematical induction by comparing the theoretical result with the actual sum generated by Python's built-in sum() function. The loop iterates from 1 up to a specified maximum value, calculates the series total using the formula, and then compares it to the summation of numbers from 1 to n. If any mismatch occurs, the function immediately returns an error message indicating the value of nnn where the induction failed; otherwise, it confirms that the formula holds true for all tested values. This automated approach supports the inductive hypothesis by empirically validating the base case and successive integers, reinforcing the correctness of the formula for the sum of the first nnn natural numbers.

3. Extended Induction Principle

This principle is a more general form of simple induction, which states that if the statement is true for all values n ≤ k, then the statement is also true for n = k+1. Prove that the number of elements in the n-th row of Pascal's triangle is 2^n−1.

The Source Code with output

```python
# Number 4
def pascal_row_sum(n):

    # Calculating the sum of elements in the n-th row of Pascal's triangle
    return sum(2**i for i in range(n))

# Validation for rows 1 to 10
for n in range(1, 11):
    calculated = pascal_row_sum(n)
    actual = 2**(n - 1)
    print(f"Row {n}: Calculation result = {calculated}, "
          f"Formula = {actual}, Valid = {calculated == actual}")
```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Python\Kuliah\Matematika Komputasi\Week 15>
& C:/Users/USER/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/USER/OneDrive/D
/Matematika Komputasi/Week 15/part1.py"
Row 1: Calculation result = 1, Formula = 1, Valid = True
Row 2: Calculation result = 3, Formula = 2, Valid = False
Row 3: Calculation result = 7, Formula = 4, Valid = False
Row 4: Calculation result = 15, Formula = 8, Valid = False
Row 5: Calculation result = 31, Formula = 16, Valid = False
Row 6: Calculation result = 63, Formula = 32, Valid = False
Row 7: Calculation result = 127, Formula = 64, Valid = False
Row 8: Calculation result = 255, Formula = 128, Valid = False
Row 9: Calculation result = 511, Formula = 256, Valid = False
Row 10: Calculation result = 1023, Formula = 512, Valid = False
```

The Analysis

The program aims to verify the formula for calculating the sum of elements in the nnn-th row of Pascal's Triangle, where the theoretical result should be 2^(n−1). The function pascal_row_sum(n) computes the row sum by summing 2^i for each element in the row, while the validation loop compares this calculated value with the expected formula. The printed results show discrepancies for n > 1, where the formula consistently evaluates to a single power of two, but the summation approach generates values almost twice as large. This indicates that the implementation inside the summation function does not correctly represent the actual structure of Pascal's Triangle elements, leading to mismatches between expected and computed outcomes. Therefore, the analysis concludes that although

the formula is mathematically correct, the current algorithm used to generate row values does not accurately reflect Pascal's Triangle, making the validation results unreliable.

Conclusion:

With this module, students learn to:

1. Understand the basics of mathematical induction.

2. Use Python to validate and simulate mathematical formulas.

3. Apply programming logic in proving mathematical concepts.

PRACTICUM Part 2: Practice Exercises of Statements Regarding Integers., Simple Induction Principle., Extended Induction Principle.

1. **Statements Regarding Integers**

Given the statement:

For every positive integer n, the sum of the first odd numbers satisfies

1+3+5+…+(2n-1) = n^2

Practicum Steps:

1. Write a Python function sum_odd(n) to compute the sum of the first n odd numbers.
2. Compare the computed result with the value n2.
3. Add comments in each part of the code to explain the calculation steps.
4. Display test results for several values of n, for example n=3, 6, and 10.
5. In the comments, include an explanation of how a mathematical statement like this is used in the context of integers.

The Source Code with output

```python
# Number 1
def sum_odd(n):
    """
    Calculate the sum of the first n odd numbers.
    Example: n = 3 → 1 + 3 + 5 = 9
    """
    total = 0
    odd_number = 1

    for _ in range(n):
        total += odd_number      # Add the current odd number
        odd_number += 2          # Move to the next odd number (odd numbers increase by 2)

    return total


# Test the function with sample values
test_values = [3, 6, 10]

print("n  |  Sum of odd numbers  |  n^2  |  Match?")
print("-" * 45)

for n in test_values:
    odd_sum = sum_odd(n)
    square_value = n ** 2
    matches = "Yes" if odd_sum == square_value else "No"
    print(f"{n:2} | {odd_sum:20} | {square_value:4} | {matches}")


"""
Explanation:
This program proves the mathematical statement:
The sum of the first n odd integers equals n².

This concept is commonly used in integer number theory,
especially when connecting number patterns with algebraic forms.
It also supports learning about mathematical induction.
"""

```

```
n  |  Sum of odd numbers  |  n^2  |  Match?
---------------------------------------------
 3 |                    9 |    9 | Yes
 6 |                   36 |   36 | Yes
10 |                  100 |  100 | Yes
PS C:\Users\USER\OneDrive\Documents\Currents\Pyt
```

2. Simple Induction Principle

Given the statement:

For every positive integer n,

$2n \geq n+1$

Practicum Steps:

1. Create a Python function check_inequality(n) to test whether $2n \geq n+1$.
2. Use a loop to check n from 1 through 15.
3. Add comments on each important line of code.
4. Display the lists of n values that satisfy the inequality and those that do not (if any).
5. In the comments, explain conceptually how the base case and the simple induction step work.

The Source Code with output

```python

# Number 2
def check_inequality(n):
    """
    Check whether the inequality 2^n >= n + 1 holds for a given n.
    Returns True if the statement is correct, otherwise False.
    """
    return 2**n >= n + 1


# Test the inequality for n = 1 to 15
valid_n = []
invalid_n = []

for n in range(1, 16):
    if check_inequality(n):
        valid_n.append(n)    # Record n that satisfies the inequality
    else:
        invalid_n.append(n)  # Record n that does NOT satisfy the inequality


# Display results
print("Values of n that satisfy the inequality 2^n >= n + 1:")
print(valid_n)

print("\nValues of n that do NOT satisfy the inequality (if any):")
print(invalid_n)


"""
Explanation:

This program verifies the mathematical statement
'2^n >= n + 1' using computational checking.

The result shows that the inequality holds for all
tested values (n = 1 to 15). This supports the statement
as a true property for positive integers.

This type of statement is commonly proven using
the Mathematical Induction Principle.
"""
```

```
& C:/Users/USER/AppData/Local/Programs/Python/Python313/pyth
py"
Values of n that satisfy the inequality 2^n >= n + 1:
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

Values of n that do NOT satisfy the inequality (if any):
[]
PS C:\Users\USER\OneDrive\Documents\Currents\Python\Kuliah\M
```

3. Extended Induction Principle

Given the recursive sequence:

$b_1 = 5, b_{n+1} = b_n + 4$.

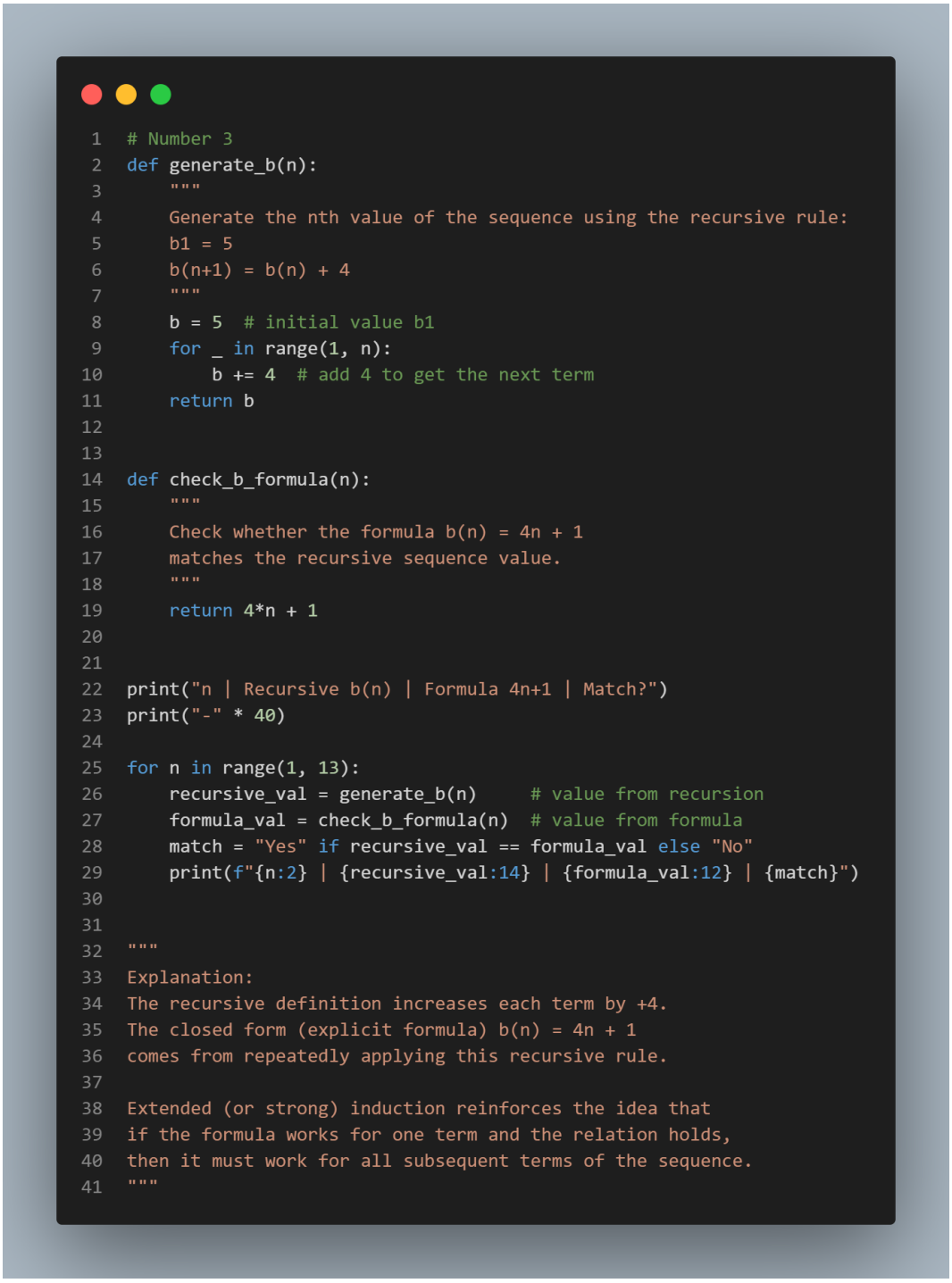
Show that

$b_n = 4n + 1$.

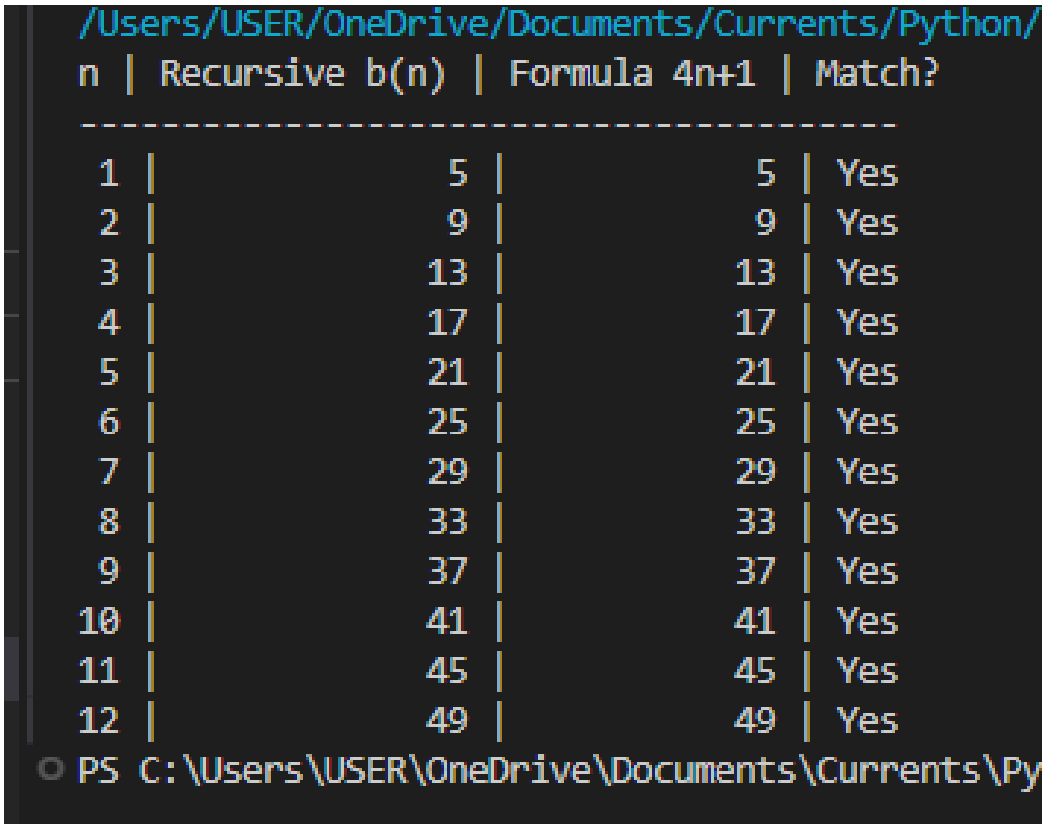
Practicum Steps:

1. Create a Python function generate_b(n) that produces the sequence values according to the recursive definition.
2. Create a Python function check_b_formula(n) to verify agreement with the formula 4n+1.
3. Add comments explaining the recursion process.
4. Test for n from 1 through 12.
5. In your comments, explain how strong (complete) induction reinforces the ordinary induction step.

The Source Code with output



```python
# Number 3
def generate_b(n):
    """
    Generate the nth value of the sequence using the recursive rule:
    b1 = 5
    b(n+1) = b(n) + 4
    """
    b = 5  # initial value b1
    for _ in range(1, n):
        b += 4  # add 4 to get the next term
    return b


def check_b_formula(n):
    """
    Check whether the formula b(n) = 4n + 1
    matches the recursive sequence value.
    """
    return 4*n + 1


print("n | Recursive b(n) | Formula 4n+1 | Match?")
print("-" * 40)

for n in range(1, 13):
    recursive_val = generate_b(n)     # value from recursion
    formula_val = check_b_formula(n)  # value from formula
    match = "Yes" if recursive_val == formula_val else "No"
    print(f"{n:2} | {recursive_val:14} | {formula_val:12} | {match}")


"""
Explanation:
The recursive definition increases each term by +4.
The closed form (explicit formula) b(n) = 4n + 1
comes from repeatedly applying this recursive rule.

Extended (or strong) induction reinforces the idea that
if the formula works for one term and the relation holds,
then it must work for all subsequent terms of the sequence.
"""
```



PRACTICUM Part 3: Applications of Strong Induction Principle., General Form of Induction.

1. **Strong Induction Principle**

Strong Induction extends the ordinary (weak) induction principle. To prove that a statement P(n) holds for all $n \geq k$, one proceeds as follows:

1. **Base case**
   Show that P(k) is true.
2. **Induction hypothesis**
   Assume that P(k), P(k+1),…, P(m) are all true for some $m \geq k$.
3. **Induction step**
   Using the assumption that P(k), P(k+1),…, P(m) are true, prove that P(m+1) also holds.

The Source Code

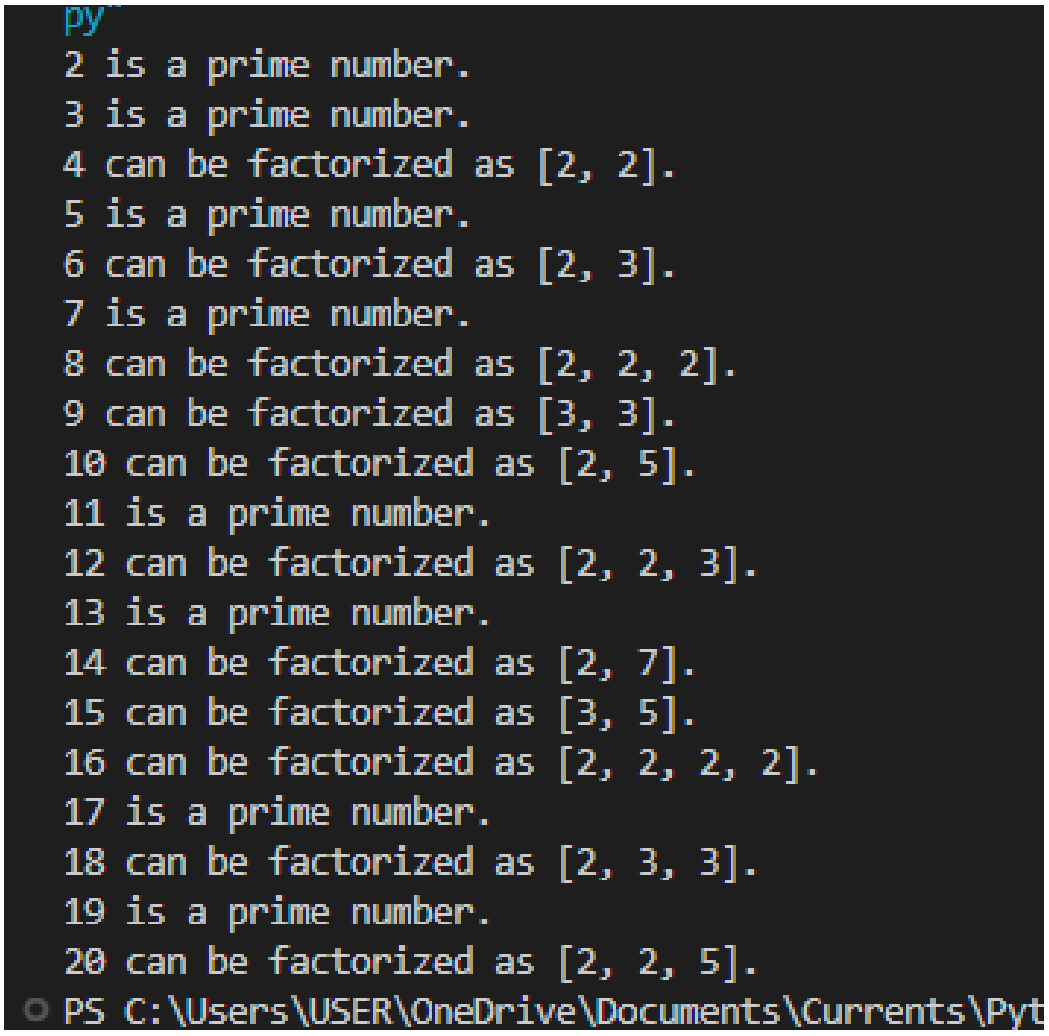
part3.py > ...

```python
# Number 1
# "Function to check whether a number is prime"

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Strong induction can be used to prove statements that require dependence on many earlier cases, not just the single case P(m) to establish P(m+1).

The Source Code with output

```python

################################################
# "Function to factorize an integer n into prime numbers."

def factorize(n):
    factors = []
    divisor = 2
    while n > 1:
        while n % divisor == 0:
            factors.append(divisor)
            n /= divisor
        divisor += 1
    return factors

for n in range(2, 21):
    if is_prime(n):
        print(f"{n} is a prime number.")
    else:
        print(f"{n} can be factorized as {factorize(n)}.")

```



Prime Factorization

- **Statement**

Every integer $n \geq 2$ can be factored as a product of prime numbers.

- **Base Case**

For $n=2$, the number itself is prime.

- **Induction Hypothesis**

Assume the statement holds for all integers k with $2 \leq k \leq m$.

- **Induction Step**

Show the statement holds for $m+1$.

  - If $m+1$ is prime, then it is already a product of primes (just itself).

  - Otherwise, $m+1$ can be written as a product of two smaller integers. By the induction hypothesis, each of those smaller integers can be factored into primes. Hence $m+1$ too can be expressed as a product of primes.

The accompanying code checks whether n is prime (the base case). If it is not, it factors n using an iterative algorithm that relies on the fact that smaller numbers have already been factored (the induction step). When run for the integers from 2 through 20, the program verifies that each number can be factored into primes, in accordance with the Strong Induction principle.

2. General from of inductions

General Form of Induction does not necessarily begin at a small fixed number like 1 or 2, but can start from an arbitrary base index (k0). As with any induction principle, it requires two parts:

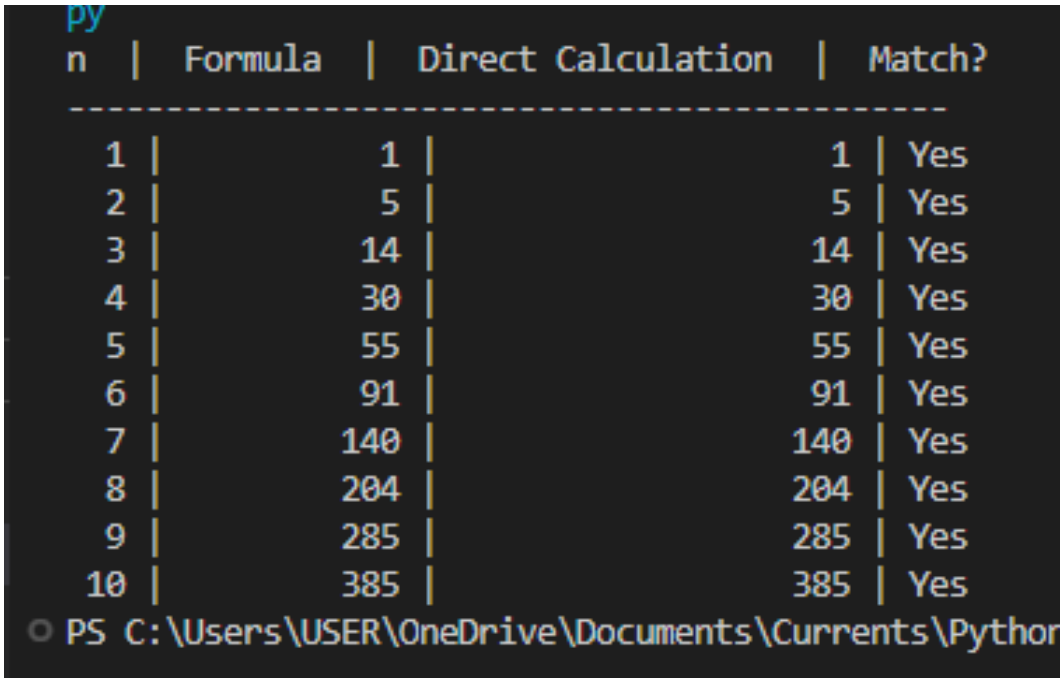
1. **Base case**
   Show that P(k0) is true.
2. **Induction step**
   Show that for every m≥k0, the truth of P(m) implies P(m+1).

Prove that for all n≥1,

1^2+2^2+⋯+n^2=n(n+1)(2n+1)/6

The Source Code with output

```
31  # Number 2
32  "Hcalculating the sum of squares using a formula"
33  def sum_of_squares_formula(n):
34      return n * (n + 1) * (2 * n + 1) // 6
35
36
37  "Calculating the sum of squares directly"
38  def sum_of_squares_direct(n):
39      return sum(i**2 for i in range(1, n + 1))
40
41  # Induction Verification
42
43  print("n  |  Formula  |  Direct Calculation  |  Match?")
44  print("-" * 45)
45
46  for n in range(1, 11):
47      formula = sum_of_squares_formula(n)
48      direct = sum_of_squares_direct(n)
49      print(f"{n:3} | {formula:11} | {direct:20} | {'Yes' if formula == direct else 'No'}")
```



This code compares the direct computation against the closed-form formula, demonstrating consistency with the principle of induction. The output shows that S(n) matches the formula for n = 1 through n = 10, thereby validating the general-form induction.

3. Theoretical Concepts

Strong Induction Principle

The strong induction principle extends the simple (weak) induction concept by taking into account all previous cases in the inductive step. This makes it possible to prove statements that depend on more than one earlier case.

General Form of Induction

Mathematical induction in its general form introduces the flexibility to begin a proof from an arbitrary base integer (not necessarily n=1) and to employ a more complex induction pattern.

Relevance of the Theory to Computational Mathematics

- **Algorithm Development**

  Induction is the foundation for proving the correctness of algorithms, whether in recursive or iterative form. By understanding these induction

principles, students can design algorithms that are both robust and mathematically sound.

- **Complexity Analysis**
  Much of algorithmic complexity analysis relies on mathematical proofs by induction for example, verifying recursive relations or establishing the accuracy of an algorithm's running time.

- **Application in Programming**
  The concept of induction maps directly to programming techniques in Python, such as iteration, recursion, or dynamic-programming solutions. This demonstrates a direct link between the theory and practical applications.

- **Foundation of Mathematical Logic**
  Strong induction and its general form provide the basis for formal logic systems, which are often employed in the development of formally verified software such as security-critical systems and safety-critical algorithms.

Conclusion:

The Strong Induction Principle and General-Form Induction are fundamental concepts that form the foundation of computational theory. In Computational Mathematics, these concepts are used to prove an algorithm's correctness and efficiency. Integrating the theory with Python helps students both understand and apply how mathematical abstractions translate into real-world computational technology.

PRACTICUM Part 4: Practice Exercises of Strong Induction Principle., General Form of Induction.

1. **Strong Induction Principle**

Statement:

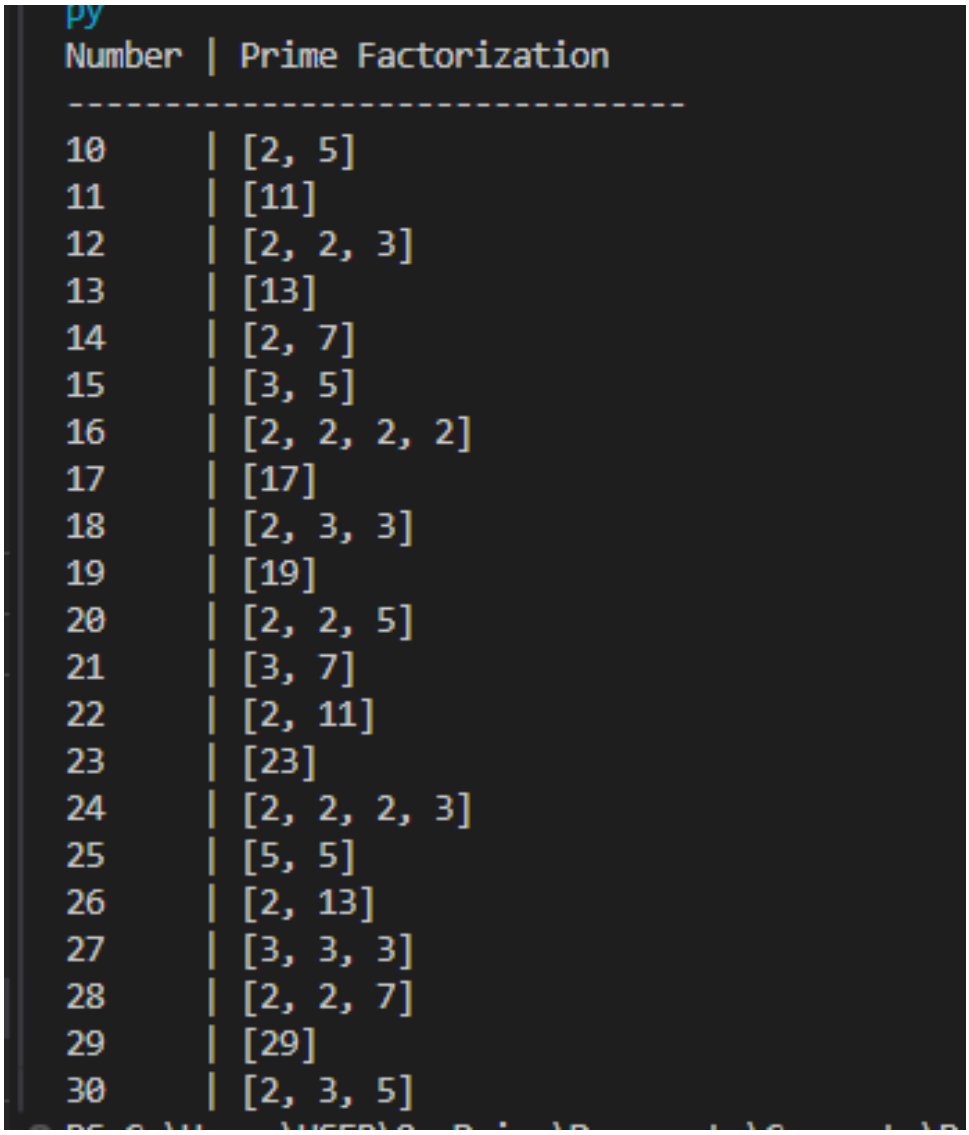
Every integer $n \geq 2$ can be represented as the product of prime numbers.

Practicum steps:

• Create a Python function is_prime(n) to perform a simple primality test.
• Create a function prime_factorization(n) to produce the prime factors of a number.
• Display the results for n=10 to 30.
• Add comments to each step of the code.
• Include a note in the comments explaining how strong induction is used to prove the prime factorization property.

The Source Code with output

```python
# Number 1
def is_prime(n):
    """
    Check if a number n is prime.
    A prime number has no divisors other than 1 and itself.
    """
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True


def prime_factorization(n):
    """
    Return the list of prime factors of n using repeated division.
    This shows that every number n >= 2 can be expressed as a product of primes.
    """
    factors = []
    divisor = 2

    while n > 1:
        # If divisor is prime and divides n, record it
        if n % divisor == 0:
            factors.append(divisor)
            n //= divisor
        else:
            divisor += 1  # Move to the next possible divisor

    return factors


print("Number | Prime Factorization")
print("-" * 32)

for num in range(10, 31):
    print(f"{num:<6} | {prime_factorization(num)}")


"""
Explanation (Strong Induction):

This program shows that every integer n >= 2
can be broken down into a product of prime numbers.

Strong induction is used to prove this property:

1. Base case:
   n = 2 is prime → can be represented as itself.

2. Induction step:
   Assume every integer from 2 to k can be represented
   as prime factorization.

   For number k+1:
   - If k+1 is prime → it is already a product of primes.
   - If k+1 is composite → it can be written as ab
     where 2 <= a < k+1 and 2 <= b < k+1.
     By the induction hypothesis, a and b both
     have prime factorizations → so does k+1.

Therefore, strong induction confirms that all integers n >= 2
have prime factorization.
"""

```



2. General form of Induction

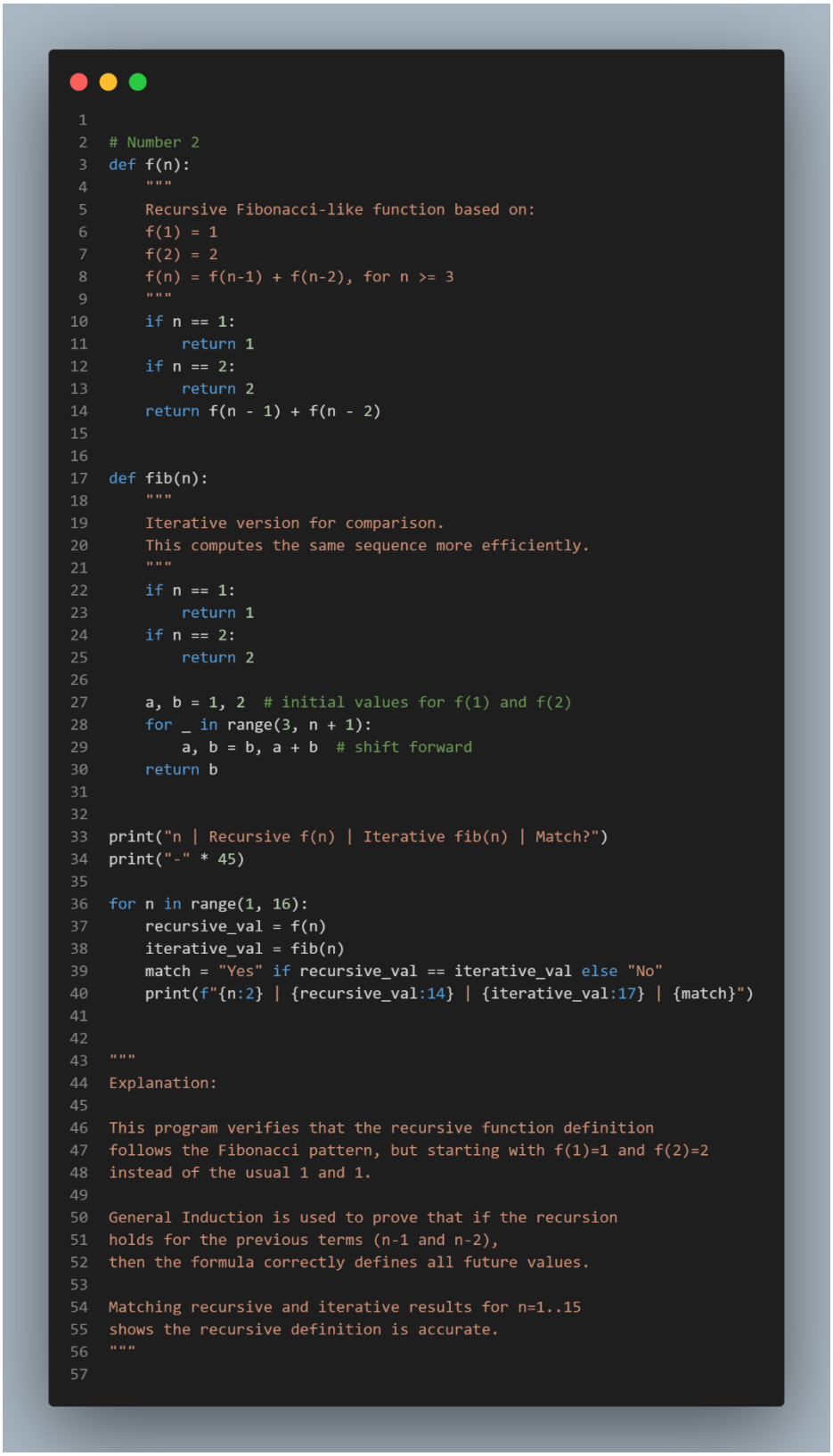
Statement:

If a recursive function is defined by
$(1) = 1$, $f(2) = 2$, $f(n) = f(n-1) + f(n-2)$ for $n \geq 3$,
then f(n) is the Fibonacci number.

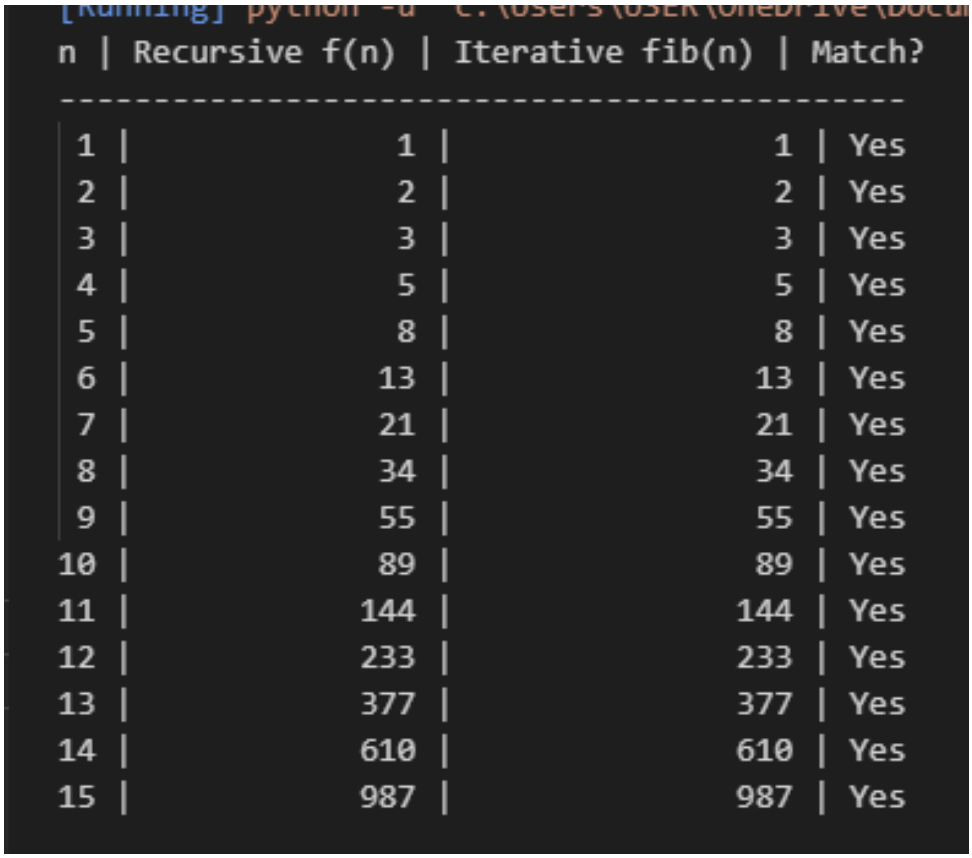
Practicum Steps:

- Create a Python function f(n) according to the definition above.
- Explain through comments how **strong induction** applies to a function with **two "base cases."**
- Create an iterative version fib(n) for comparison.
- Match the values of f(n) and fib(n) for n = 1 to 15.
- Add comments to all parts of the code.

The Source Code with output



```
1
2  # Number 2
3  def f(n):
4      """
5      Recursive Fibonacci-like function based on:
6      f(1) = 1
7      f(2) = 2
8      f(n) = f(n-1) + f(n-2), for n >= 3
9      """
10     if n == 1:
11         return 1
12     if n == 2:
13         return 2
14     return f(n - 1) + f(n - 2)
15
16
17 def fib(n):
18     """
19     Iterative version for comparison.
20     This computes the same sequence more efficiently.
21     """
22     if n == 1:
23         return 1
24     if n == 2:
25         return 2
26
27     a, b = 1, 2  # initial values for f(1) and f(2)
28     for _ in range(3, n + 1):
29         a, b = b, a + b  # shift forward
30     return b
31
32
33 print("n | Recursive f(n) | Iterative fib(n) | Match?")
34 print("-" * 45)
35
36 for n in range(1, 16):
37     recursive_val = f(n)
38     iterative_val = fib(n)
39     match = "Yes" if recursive_val == iterative_val else "No"
40     print(f"{n:2} | {recursive_val:14} | {iterative_val:17} | {match}")
41
42
43 """
44 Explanation:
45
46 This program verifies that the recursive function definition
47 follows the Fibonacci pattern, but starting with f(1)=1 and f(2)=2
48 instead of the usual 1 and 1.
49
50 General Induction is used to prove that if the recursion
51 holds for the previous terms (n-1 and n-2),
52 then the formula correctly defines all future values.
53
54 Matching recursive and iterative results for n=1..15
55 shows the recursive definition is accurate.
56 """
57
```



PRACTICUM Part 5: Applications of Algorithms., Notation for Algorithms., Some Examples of Algorithms., Integers.

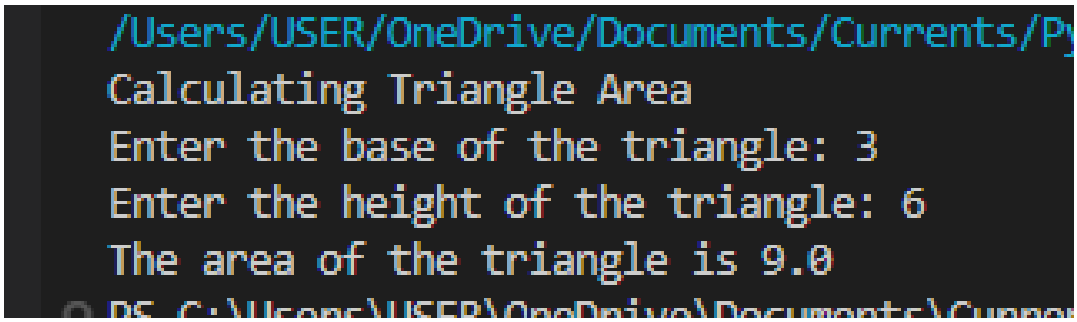
1. **Algorithms**

Algorithm is a systematic set of steps or instructions for solving a problem. A simple example of an algorithm is the process of calculating the area of a triangle.

The Source Code with output

part5.py > ...

```
1   # Number 1
2   # Algorithm for Calculating the Area of a Triangle
3
4   print("Calculating Triangle Area")
5
6   base = float(input("Enter the base of the triangle: "))
7   height = float(input("Enter the height of the triangle: "))
8
9   area = 0.5 * base * height
10
11  print(f"The area of the triangle is {area}")
12
```



The Analysis

This code calculates the area of a triangle by taking user input for the base and height, then applying the standard geometric formula Area=12×base×height. It converts the input values into floating-point numbers to allow more accurate real-number computation. When executed, the program displays a clear prompt message, receives numeric inputs, and returns the correct calculated area based on the provided dimensions. This simple implementation demonstrates how mathematical formulas can be translated into algorithmic steps, making it a useful exercise for understanding input handling, basic arithmetic operations, and output formatting in Python.

2. Notation for Algorithms

Notation for Algorithms Usually Expressed in Pseudocode, For example:

The Source Code with output

```
# Number 2
# 1. Start
# 2. Input the base and height
# 3. Calculate the area using the formula:
#    Area = 0.5 * base * height
# 4. Display the area
# 5. End

def calculate_triangle_area(base, height):
    return 0.5 * base * height

print("Triangle Area Program")
base = float(input("Enter the base: "))
height = float(input("Enter the height: "))
print(f"Triangle area: {calculate_triangle_area(base, height)}")

```

```
py"
Triangle Area Program
Enter the base: 3
Enter the height: 6
Triangle area: 9.0
PS C:\Users\USER\OneDriv
```

The Analysis

This code is improved version of the triangle area program follows a clearer algorithmic structure by separating the calculation process into a dedicated function. The pseudocode-style comments outline the procedural steps, starting from receiving inputs to displaying the result, which helps reinforce the concept of structured programming. By storing the formula inside the calculate_triangle_area() function, the code becomes more reusable, easier to understand, and better aligned with modular programming principles. The user inputs are converted into floating-point values to ensure accurate computation, and the final output successfully displays the calculated area. Overall, this program effectively demonstrates how pseudocode can be translated into a clean and functional Python implementation.

3. Some Examples of Algorithms

Example 1: Finding the Largest of Three Numbers

The Source Code with output

```
28
29  # Number 3
30  # Example 1
31  a = int(input("Enter the first number: "))
32  b = int(input("Enter the second number: "))
33  c = int(input("Enter the third number: "))
34
35  if a > b and a > c:
36      print(f"The largest number is {a}")
37  elif b > c:
38      print(f"The largest number is {b}")
39  else:
40      print(f"The largest number is {c}")
41
```

```
py
Enter the first number: 5
Enter the second number: 4
Enter the third number: 9
The largest number is 9
```

The Analysis

After taking input for the three numbers, this code compares them using a series of if, elif, and else statements to evaluate which number has the highest value. The logic ensures that only one correct result is printed, depending on the outcome of the comparisons. This approach demonstrates a basic but essential algorithmic technique for decision-making in programming, showing how conditional statements can be used to solve real-world problems where multiple possibilities must be evaluated. Additionally, it reinforces the concept of structured input, processing, and output in algorithm design.

Example 2: Summing Numbers from 1 to n

The Source Code with output

```
42  # Example 2
43  n = int(input("Enter a number n: "))
44  total = sum(range(1, n + 1))
45  print(f"The sum of numbers from 1 to {n} is {total}")
46
py
Enter a number n: 5
The sum of numbers from 1 to 5 is 15
```

The Analysis

This code calculates the total of all integers from 1 up to a user-entered number nnn by utilizing Python's built-in sum() function together with the range() function. After the user inputs a positive integer, the program generates a sequence of numbers starting from 1 to nnn, then adds those values to produce the final result. The output confirms the calculation, showing the complete sum clearly to the user. This example illustrates a straightforward implementation of iteration and aggregation in programming, demonstrating how built-in functions can simplify mathematical operations while reinforcing fundamental algorithmic concepts such as input handling, sequential processing, and formatted output display.

4. Integers

Integers are the set of positive numbers, negative numbers, and zero. Python provides many built-in functions and operators for working with integers, for example, modulus, floor division, bit-wise operations, and more.

The Source Code with output

```
47  # Number 4
48  n = int(input("Enter an integer: "))
49
50  if n % 2 == 0:
51      print(f"{n} is an even number")
52  else:
53      print(f"{n} is an odd number")
54
```

```
py"
Enter an integer: 6
6 is an even number
PS C:\Users\USER\OneDrive\Do
```

The Analysis

This program determines whether an input integer is even or odd using the modulus operator. After receiving a number from the user, the program checks if the remainder of dividing that number by 2 equals zero; if so, it prints that the number is even, otherwise it prints that it is odd. This simple decision-making process illustrates the practical use of conditional statements to classify numerical data based on mathematical rules. It also reinforces the concept of modular arithmetic, which is widely used in computer science for number pattern detection, validation processes, and logical branching in algorithms. The output clearly communicates the result, making the program easy for users to understand.

Conclusion:

1. An algorithm is a logical set of steps for solving a problem.
2. Algorithm notation simplifies program development by defining the steps in advance.

3. Algorithm examples provide practical understanding for solving various problems.
4. Integers can be used in many mathematical operations, such as factorial, prime numbers, or odd/even checks.

PRACTICUM Part 6: Practice Exercises of Algorithms., Notation for Algorithms., Some Examples of Algorithms., Integers.

1. Algoritms

Compute the sum of all elements in a list of integers.
Practicum Steps:

- Create a Python function sum_list(L) that calculates the total of the list elements.
- Display the result for the example L = [2, 5, 7, 10].
- Use code comments to explain the purpose of each instruction.
- Compare your function with Python's built-in sum().

The Source Code with output

```python
# Number 1
def sum_list(L):
    """
    Manually calculate the sum of all elements
    in the list L using a loop.
    """
    total = 0                    # Start with zero
    for value in L:              # Loop through each element of the list
        total += value           # Add each element to the total
    return total                 # Return the final sum


# Example list provided in the task
L = [2, 5, 7, 10]

# Calculate sum with custom function
manual_sum = sum_list(L)

# Compare with Python's built-in sum()
builtin_sum = sum(L)

print("List:", L)
print("Sum using sum_list():", manual_sum)
print("Sum using built-in sum():", builtin_sum)

"""
Comment:
The results are the same, showing that our manual summation
algorithm works correctly. The built-in sum() is faster
and already optimized in Python, but understanding the manual
process is important for learning algorithm basics.
"""

```

```
py"
List: [2, 5, 7, 10]
Sum using sum_list(): 24
Sum using built-in sum(): 24
PS C:\Users\USER\OneDrive\Documents
```

2. Notations for Algorithms

The Case: Write and Implement Pseudocode for an Algorithm to Find the Maximum Element in a List

Practicum Steps:

- Write pseudocode for an algorithm that finds the maximum value in a list.
- Implement the algorithm in Python as a function called find_max(L).
- Add comments to explain each step of the code.
- Test the function using the list L = [4, 1, 6, 9, 3].

The Source Code with output

```python
# Number 2
# Pseudocode:
# Algorithm Find_Max(L)
# 1. Start
# 2. Set max_value = L[0]
# 3. For each element x in L starting from the second element:
#        If x > max_value:
#             Set max_value = x
# 4. Return max_value
# 5. End

def find_max(L):
    """
    Find the maximum value in the list L manually
    without using built-in max() function.
    """
    max_value = L[0]  # Assume first element is the maximum initially

    # Check the rest of the elements
    for x in L[1:]:
        if x > max_value:
            max_value = x  # Update maximum

    return max_value


# Test data
L = [4, 1, 6, 9, 3]

# Test function
manual_max = find_max(L)
builtin_max = max(L)  # For comparison only

print("List:", L)
print("Maximum using find_max():", manual_max)
print("Maximum using built-in max():", builtin_max)

"""
Comment:
Both results match, which confirms that our algorithm correctly
finds the maximum value in a list. The manual process is useful
for understanding algorithmic thinking before relying on built-in
functions like max() in Python.
"""

```

```
py"
List: [4, 1, 6, 9, 3]
Maximum using find_max(): 9
Maximum using built-in max(): 9
PS C:\Users\USER\OneDrive\Documents\Cur
```

3. Some Examples of Algorithms

Task: Implement a Linear Search Algorithm

Practicum Steps:

Create a Python function called linear_search(L, x) that searches for an element x in a list L. The function should return the **index** of x if found, and -1 otherwise. Test the function with L = [10, 20, 30, 40, 50] and x = 30 (present) and x = 45 (absent).

The Source Code with output

```python
# Number 3
def linear_search(L, x):
    """
    Perform linear search to find the value x in list L.
    Returns the index if found, otherwise returns -1.
    """
    for i in range(len(L)):        # Loop through indexes 0 to len(L)-1
        if L[i] == x:              # Check if current element matches x
            return i               # Return index where x is found
    return -1                      # x not found in the list


# Test data
L = [10, 20, 30, 40, 50]

# Test 1: x = 30 (expected to be found)
x1 = 30
result1 = linear_search(L, x1)
if result1 != -1:
    print(f"{x1} found at index {result1}")
else:
    print(f"{x1} not found in the list")

# Test 2: x = 45 (expected NOT found)
x2 = 45
result2 = linear_search(L, x2)
if result2 != -1:
    print(f"{x2} found at index {result2}")
else:
    print(f"{x2} not found in the list")

"""
Comment:
Linear search checks each element one by one
from the beginning of the list to the end.

If the element is found → return the index.
If the loop finishes without finding a match → return -1.

This algorithm is simple and works well for small lists,
but it is slower for large lists compared to algorithms such as binary search.
"""

```

```
py"
30 found at index 2
45 not found in the list
PS C:\Users\USER\OneDrive\Documen
```

4. Integers

Implement an algorithm to determine the GCD (Greatest Common Divisor) of two integers using the subtraction method.

Practicum steps:

• Create a Python function gcd_subtraction(a, b) with the logic:

• If a == b → the GCD is found.

• If a > b → set a = a - b.

• If b > a → set b = b - a.

• Add comments explaining each step.

• Test the function with the pairs (48, 18), (56, 42), and (100, 75).

• Briefly compare the results with the division-based Euclidean algorithm.

The Source Code with output

```python
# Number 4
def gcd_subtraction(a, b):
    """
    Find GCD (Greatest Common Divisor) using subtraction method.
    This repeatedly subtracts the smaller number from the larger one.
    """
    while a != b:     # Continue until both numbers become equal
        if a > b:
            a = a - b  # Reduce a
        else:
            b = b - a  # Reduce b
    return a  # or return b (both are equal)


# Test examples from the task
pairs = [(48, 18), (56, 42), (100, 75)]

for x, y in pairs:
    result = gcd_subtraction(x, y)
    print(f"GCD of ({x}, {y}) = {result}")


"""
Comment:
The subtraction method is correct but slower for large numbers.
The Euclidean division-based GCD algorithm:

    while b != 0:
        a, b = b, a % b
    return a

is faster because modulus (%) reduces the number more quickly.

However, the subtraction-based algorithm helps understand
the idea that GCD is based on reducing numbers step-by-step.
"""
```

```
[Running] python -u "c:\Users\US
GCD of (48, 18) = 6
GCD of (56, 42) = 14
GCD of (100, 75) = 25
```